Resumo executivo

SauceDemo (Swag Labs) · verificação em 15/10/2026 · **24 bugs confirmados** (8 afetam todos os usuários e 16 são defeitos de treino dos perfis especiais), cada um com issue no GitHub; 23 com teste automatizado (`xfail` que avisa quando o bug for corrigido) e 1 sem teste, de propósito (bug de tempo, com script de reprodução).

Bugs por usuário e severidade

Usuário	Crítica	Alta	Média	Baixa	Total
Todos (inclusive `standard_user`)	–	–	4	4	**8**
`problem_user`	1	1	4	1	**7**
`error_user`	1	1	3	–	**5**
`visual_user`	–	1	1	2	**4**
Total	**2**	**3**	**12**	**7**	**24**

Os 3 mais relevantes

1. **BUG-019** (issue #20), defeito de treino do `visual_user`: o cliente vê um preço e paga outro: os 6 preços do inventário mudam a cada carregamento e nenhum coincide com o cobrado. É um defeito que não aparece em testes funcionais comuns e só foi detectado comparando com o `standard_user` como oráculo.
2. **BUG-001** (issue #2), afeta todos os usuários: até o `standard_user` conclui um pedido de $0.00 com o carrinho vazio. Afeta o fluxo principal de todos os usuários.
3. **BUG-018** (issue #19), defeito de treino do `error_user`: o `error_user` não consegue finalizar a compra; o clique em Finish gera um erro JavaScript e nada acontece.

Onde encontrar

- **Relatório completo:** nas páginas seguintes deste PDF e no GitHub, com passos para reproduzir e evidências de cada bug.
- **Issues:** todos os bugs com a label `bug`; cada linha da tabela-resumo do relatório também aponta para a sua issue.
- **Testes:** `pytest -m known_bug` roda só os testes que documentam estes bugs.

Relatório de bugs: SauceDemo (Swag Labs)

Bugs encontrados por engenharia reversa do SauceDemo. As regras de negócio usadas como referência estão em requisitos-inferidos.md. As sugestões de melhoria estão em sugestoes-de-melhoria.md.

Sumário

- Resumo
- Ambiente e critérios
- Bugs que afetam todos os usuários
- Bugs do problem_user
- Bugs do error_user
- Bugs do visual_user
- Suspeitas
- Comportamento intencional do site de demonstração

Resumo

24 bugs confirmados: 23 novos e 1 que a suíte já cobria (BUG-007). **8 afetam todos os usuários**, inclusive o `standard_user` (BUG-001 a BUG-006, BUG-023 e BUG-024); os **16 restantes são os defeitos de treino** que o SauceDemo injeta de propósito no `problem_user`, no `error_user` e no `visual_user` (veja "Classificação" em Ambiente e critérios). Cada um foi reproduzido em pelo menos duas execuções independentes. 23 viraram um teste automatizado `xfail(strict=True)`, que falha somente pelo defeito descrito; o BUG-023, um bug de tempo, ficou sem teste de propósito e tem um script de reprodução controlada.

Severidade	Qtd.		Usuário afetado	Qtd.
Crítica	2		Todos (inclusive `standard_user`)	8
Alta	3		`problem_user`	7
Média	12		`error_user`	5
Baixa	7		`visual_user`	4
Total	**24**		`performance_glitch_user` e `locked_out_user`	0

ID	Título	Usuário	Severidade	Prioridade	Issue
BUG-001	Pedido é concluído com o carrinho vazio	todos	Média	P2	#2
BUG-002	Checkout aceita campos preenchidos só com espaços	todos	Média	P2	#3
BUG-003	Reset App State deixa os botões em "Remove"	todos	Baixa	P3	#4
BUG-004	Produto inexistente entra no carrinho e é contado	todos	Média	P3	#5
BUG-005	Rotas internas respondem HTTP 404	todos	Baixa	P3	#6
BUG-006	Telemetria com credenciais de exemplo gera erros 401	todos	Baixa	P4	#7
BUG-023	Menu lateral abre invisível se clicado logo após a página carregar	todos	Baixa	P4	#25
BUG-024	Resumo do checkout abre pela URL e conclui o pedido sem o passo 1	todos	Média	P2	#35
BUG-007	Todos os produtos exibem a imagem de erro	problem	Média	P3	#8
BUG-008	Ordenação não muda a lista	problem	Média	P2	#9
BUG-009	Sobrenome digitado vai para o campo de nome	problem	**Crítica**	P1	#10
BUG-010	Adicionar falha em 3 produtos e remover falha nos outros 3	problem	Alta	P1	#11
BUG-011	Detalhe abre o produto seguinte ao clicado	problem	Média	P2	#12
BUG-012	Link About aponta para uma página 404	problem	Baixa	P3	#13
BUG-013	Carrinho continua cheio depois do pedido	problem	Média	P2	#14
BUG-014	Ordenação exibe "Sorting is broken!" e não ordena	error	Média	P2	#15
BUG-015	Adicionar falha em 3 produtos e remover falha nos outros 3, com erro JavaScript	error	Alta	P1	#16
BUG-016	Campo de sobrenome não aceita texto	error	Média	P2	#17
BUG-017	Checkout avança sem sobrenome	error	Média	P2	#18
BUG-018	Finish não conclui o pedido	error	**Crítica**	P1	#19

ID	Título	Usuário	Severidade	Prioridade	Issue
BUG-019	Preços do inventário aleatórios e diferentes do cobrado	visual	Alta	P1	#20
BUG-020	Backpack exibe a imagem de erro	visual	Baixa	P3	#21
BUG-021	Ícone do carrinho deslocado sobre a ordenação	visual	Média	P3	#22
BUG-022	Botão do último produto desalinhado	visual	Baixa	P4	#23

Os 3 mais relevantes:

1. **BUG-019**, defeito de treino do `visual_user`: o cliente vê um preço e paga outro: os 6 preços do inventário mudam a cada carregamento e nenhum coincide com o cobrado. É um defeito que não aparece em testes funcionais comuns e só foi detectado comparando com o `standard_user` como oráculo.
2. **BUG-001**, afeta todos os usuários: até o `standard_user` conclui um pedido de $0.00 com o carrinho vazio. Afeta o fluxo principal de todos os usuários.
3. **BUG-018**, defeito de treino do `error_user`: o `error_user` não consegue finalizar a compra; o clique em Finish gera um erro JavaScript e nada acontece.

Ambiente e critérios

Item	Valor
Aplicação	https://www.saucedemo.com (produção)
Data	15/10/2026
Navegador	Chromium 153.0.8010.12 (Playwright 1.63.0), 1280×720
Sistema	Ubuntu 26.04.1 LTS, Python 3.14.4
Senha	`secret_sauce` para todos os usuários

Severidade

Nível	Critério
Crítica	Impede concluir a compra.
Alta	Função principal com resultado errado (produto ou preço errado, item que não entra ou não sai do carrinho).
Média	Validação ausente ou função secundária quebrada.
Baixa	Problema visual, técnico ou de infraestrutura sem impacto direto na compra.

Prioridade: P1 corrigir já; P2 próxima entrega; P3 planejar; P4 quando houver oportunidade.

Classificação. O SauceDemo injeta defeitos de propósito em alguns perfis para treino de automação. Mesmo assim, esses defeitos são registrados aqui como **bugs confirmados**: do ponto de vista do produto, divergem do comportamento correto (o do `standard_user`), e a suíte já trata assim o bug de imagens do `problem_user`. A categoria "comportamento intencional" fica para o que **não** é defeito, como o atraso proposital do `performance_glitch_user`.

Rodar só os testes destes bugs: `pytest -m known_bug`

Bugs que afetam todos os usuários

BUG-001: Pedido é concluído com o carrinho vazio

Usuário	Severidade	Prioridade	Teste
todos (`standard_user`)	Média	P2	`tests/test_cart_checkout.py::test_nao_conclui_pedido_com_carrinho_vazio`

Pré-condições: usuário logado, carrinho vazio.

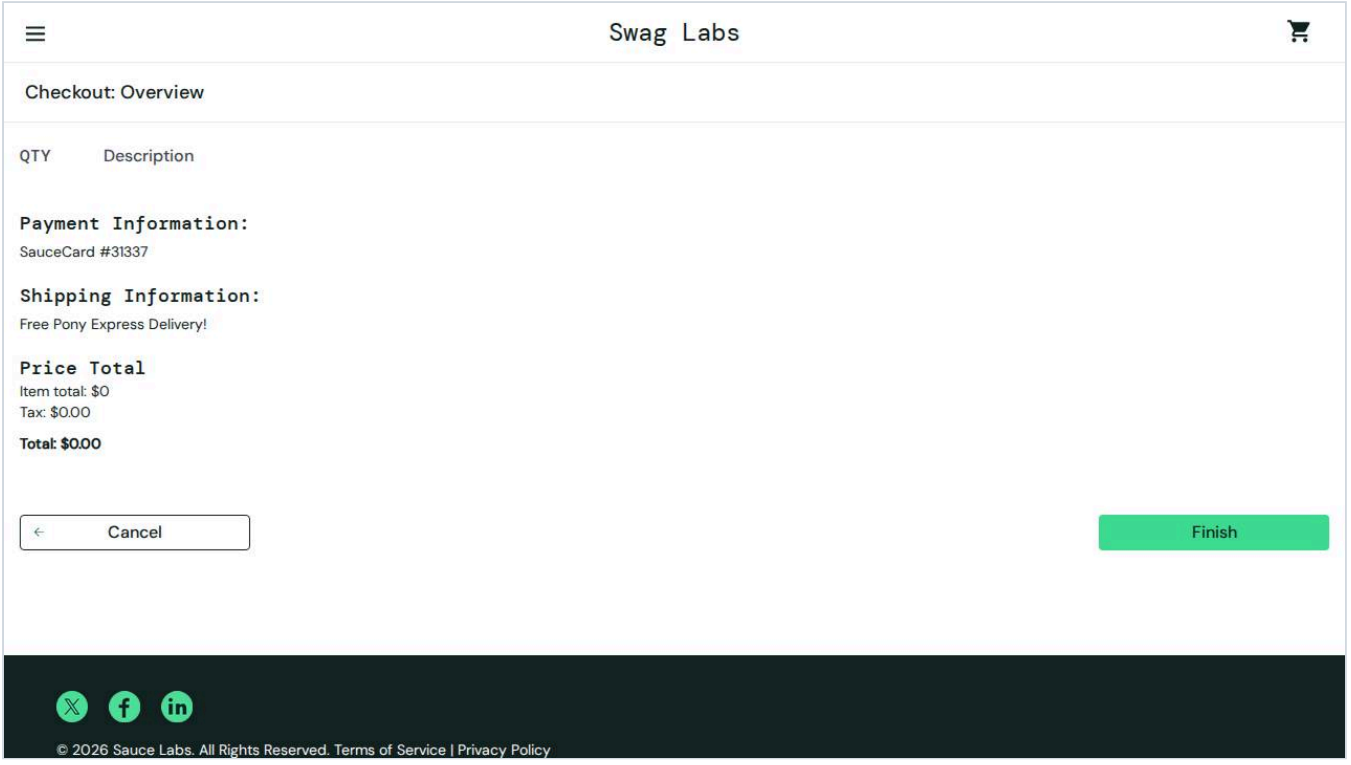
Passos

1. Abrir o carrinho (`/cart.html`) sem itens.
2. Clicar em **Checkout**.
3. Preencher nome, sobrenome e CEP e clicar em **Continue**.
4. Clicar em **Finish**.

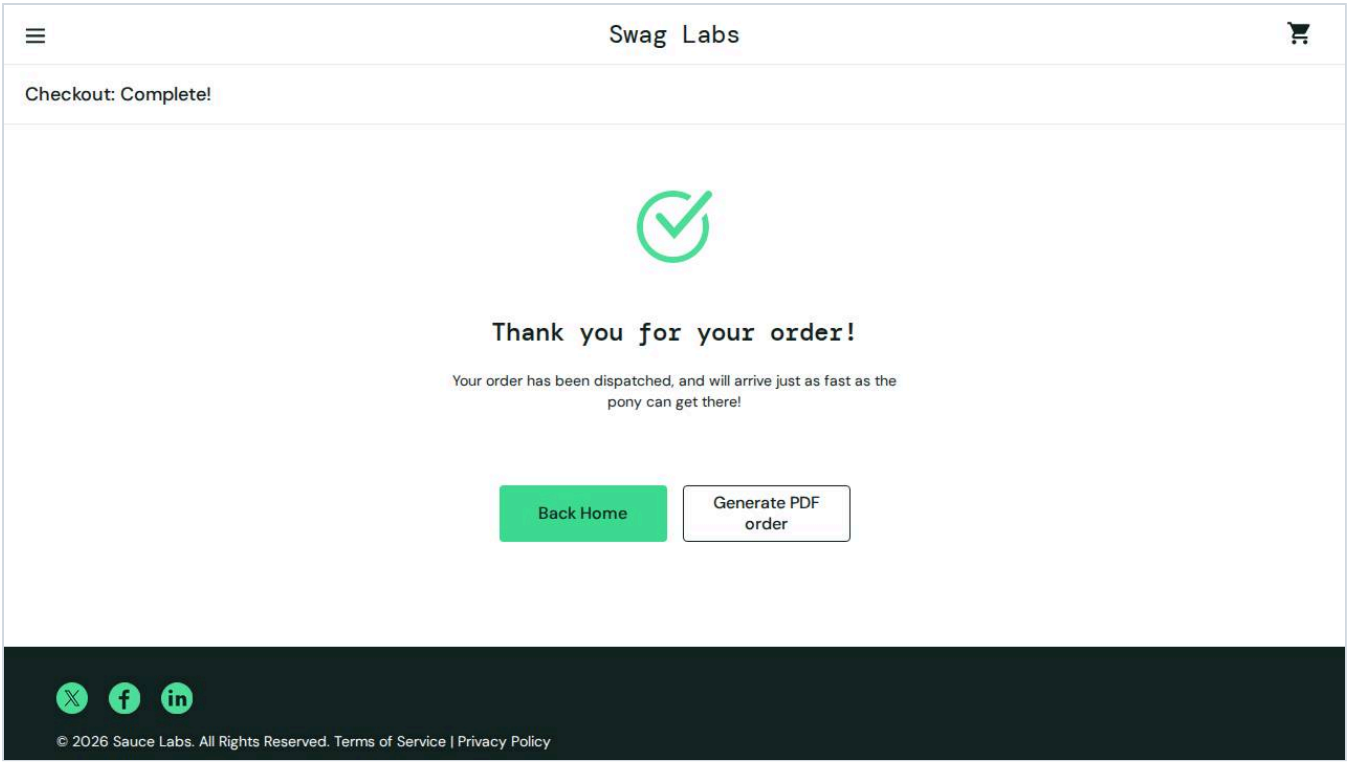
Esperado: o checkout não deve ser oferecido com o carrinho vazio, ou deve ser bloqueado com uma mensagem.

Obtido: o resumo mostra "Item total: $0" e "Total: $0.00", e o pedido é concluído com "Thank you for your order!".

Evidência: resumo vazio · pedido concluído



Evidência: resumo vazio



Evidência: pedido concluído

BUG-002: Checkout aceita campos preenchidos só com espaços

Usuário	Severidade	Prioridade	Teste
todos (`standard_user`)	Média	P2	`tests/test_cart_checkout.py::test_checkout_rejeita_campos_so_com_espacos` (3 casos)

Pré-condições: um item no carrinho, no passo 1 do checkout.

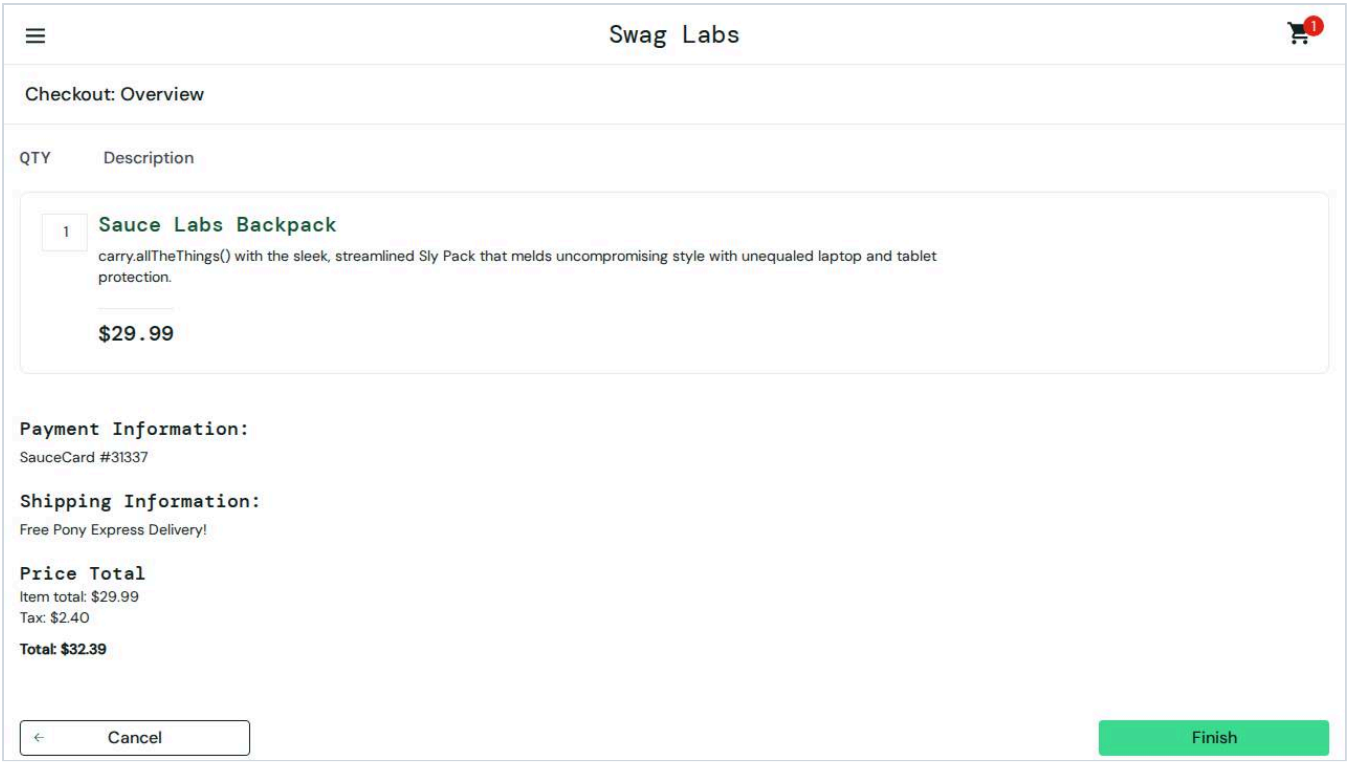
Passos

1. Preencher um dos campos (nome, sobrenome ou CEP) apenas com espaços e os outros com valores válidos.
2. Clicar em **Continue**.

Esperado: a mesma mensagem de campo obrigatório exibida para campo vazio ("Error: First Name is required", etc.).

Obtido: o formulário avança para o resumo nos três casos; a validação não ignora espaços.

Evidência: os três campos com espaços, aceitos



Evidência: os três campos com espaços, aceitos

BUG-003: Reset App State deixa os botões em "Remove"

Usuário	Severidade	Prioridade	Teste
todos (`standard_user`)	Baixa	P3	`tests/test_inventory.py::test_reset_app_state_restaura_os_botoes`

Pré-condições: dois produtos no carrinho, no inventário.

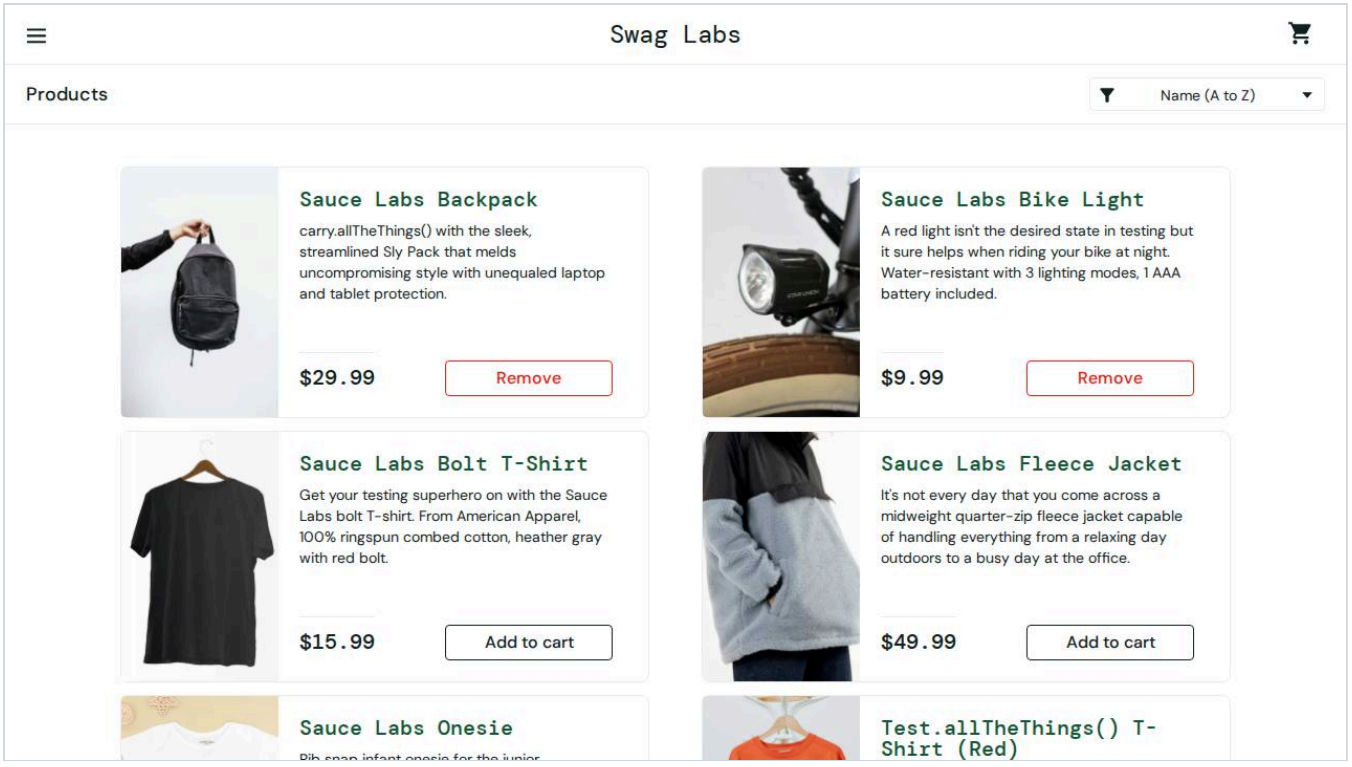
Passos

1. Abrir o menu e clicar em **Reset App State**.
2. Fechar o menu.

Esperado: contador some e os botões voltam a "Add to cart".

Obtido: o contador some e o `localStorage` é limpo, mas os botões continuam em "Remove" até recarregar a página. A tela mostra um estado que não existe mais.

Evidência: botões em Remove sem contador



Evidência: botões em Remove sem contador

BUG-004: Produto inexistente entra no carrinho e é contado

Usuário	Severidade	Prioridade	Teste
todos (`standard_user`)	Média	P3	`tests/test_product_detail.py::test_produto_inexistente_nao_entra_no_carrinho` (ids 6 e -1) e `::test_carrinho_ignora_ids_inexistentes_no_localstorage`

Pré-condições: usuário logado, carrinho vazio.

Passos (pela URL)

1. Abrir `/inventory-item.html?id=6` (também `-1` ou `999`).
2. Na página "ITEM NOT FOUND" (preço `$√-1`), clicar em **Add to cart**.
3. Abrir o carrinho.

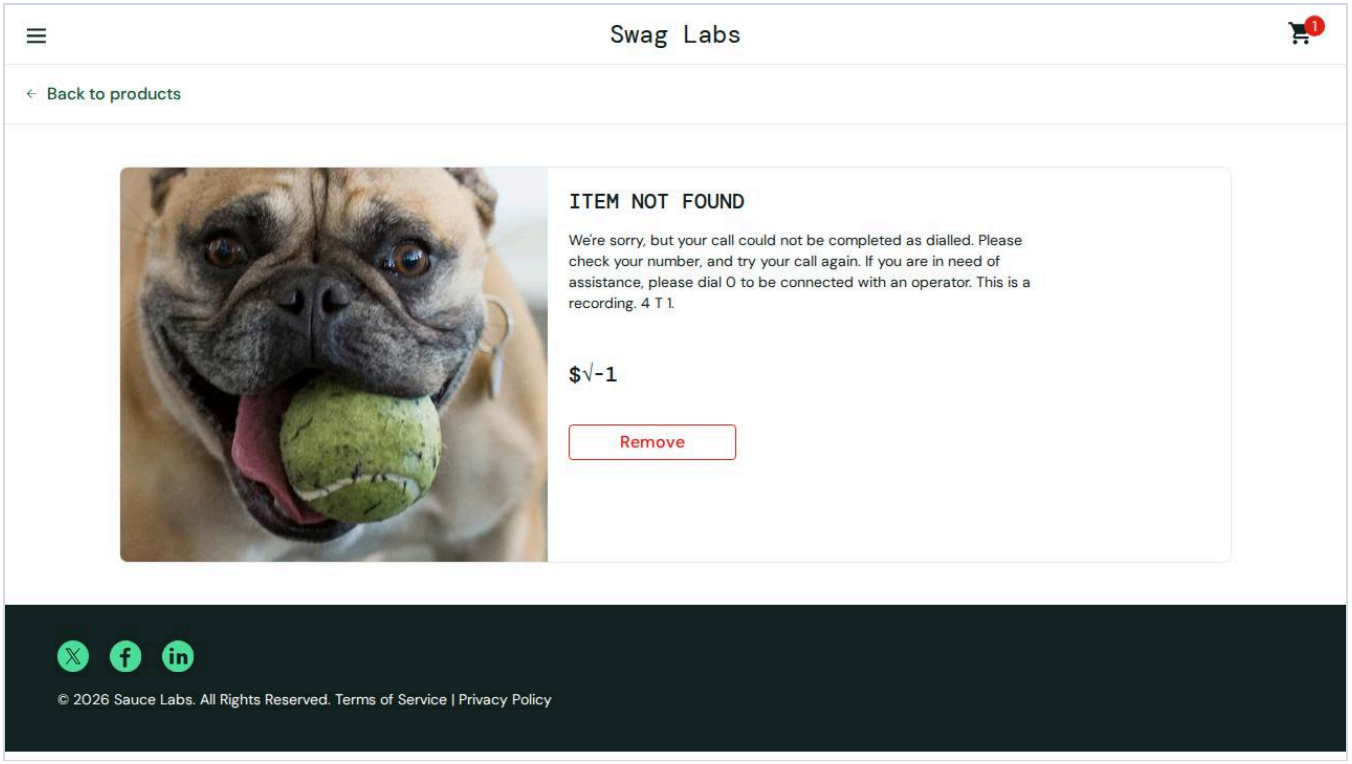
Passos (pelo localStorage)

1. Definir `cart-contents` como `[999]` e recarregar o inventário.

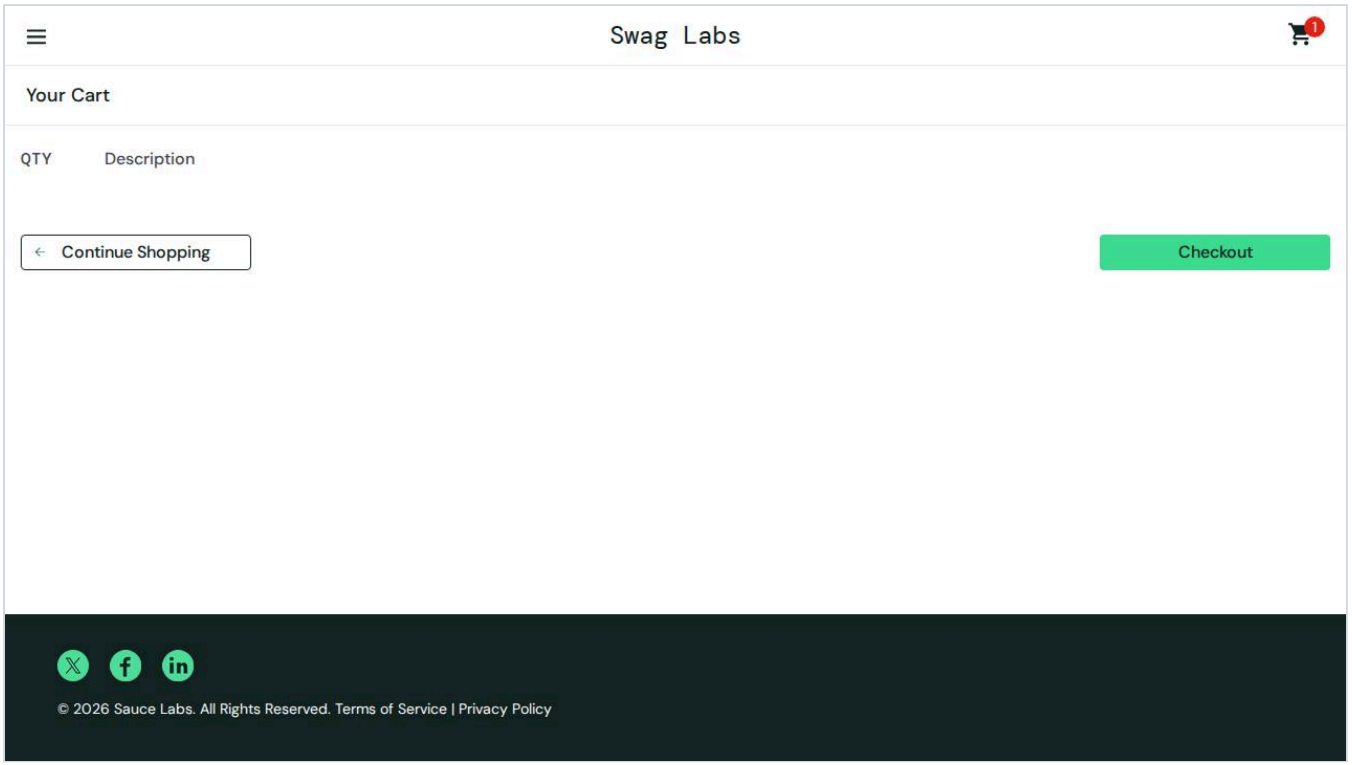
Esperado: um produto inexistente não pode ser adicionado; ids desconhecidos no carrinho devem ser descartados.

Obtido: o contador mostra **1**, mas o carrinho fica vazio; o checkout segue até um resumo sem subtotal, imposto nem total.

Evidência: produto inexistente adicionado · contador 1 com carrinho vazio



Evidência: produto inexistente adicionado



Evidência: contador 1 com carrinho vazio

BUG-005: Rotas internas respondem HTTP 404

Usuário	Severidade	Prioridade	Teste
todos	Baixa	P3	`tests/test_console_e_rede.py::test_rotas_internas_respondem_http_200`

Passos: fazer uma requisição HTTP direta (sem JavaScript) para `/inventory.html`, `/cart.html`, `/checkout-step-one.html` ou `/inventory-item.html?id=4`.

Esperado: HTTP 200 para rotas válidas da aplicação.

Obtido: HTTP **404** em todas; só `/` responde 200. A página aparece porque o JavaScript a monta mesmo assim, mas monitoramento, buscadores, ferramentas de link-check e navegadores sem JavaScript tratam as rotas como inexistentes.

Evidência: logs-rede-console-e-estado.json (chave "BUG-005")

BUG-006: Telemetria com credenciais de exemplo gera erros 401

Usuário	Severidade	Prioridade	Teste
todos	Baixa	P4	`tests/test_console_e_rede.py::test_telemetria_nao_gera_erros_http`

Passos: abrir o site em uma sessão nova do navegador, com o console aberto.

Esperado: a telemetria (Backtrace) é enviada com sucesso ou fica desligada, sem erros.

Obtido: as chamadas vão para `events.backtrace.io/...?universe=UNIVERSE&token=TOKEN` (valores literais de exemplo), recebem **401 Unauthorized** e geram erros no console. Os eventos saem uma vez por sessão do navegador, na primeira carga. Com o `error_user` e o `locked_out_user` há também erros de CORS em `submit.backtrace.io`. Os erros reportados pelo próprio site, como o do BUG-014, nunca chegam ao destino.

Evidência: logs-rede-console-e-estado.json (chave "BUG-006")

Atualização (27/10/2026): o comportamento do Backtrace mudou e o status deixou de ser observável no navegador. As duas chamadas continuam saindo na primeira carga, com `UNIVERSE`/`TOKEN`, mas o servidor leva de 8 a 10 s só no handshake TLS, e o cliente de telemetria do site aborta cada chamada em 15 s (`net::ERR_ABORTED`), sem receber status, e tenta de novo depois de 10 s. O defeito continua: as credenciais são de exemplo e nenhum evento chega ao destino. O teste trata a falta de resposta como inconclusiva (skip) em vez de marcar o bug como corrigido; volta a dar XFAIL quando o servidor responder dentro do limite.

BUG-023: Menu lateral abre invisível se clicado logo após a página carregar

Usuário	Severidade	Prioridade	Teste
todos (`standard_user`)	Baixa	P4	Sem teste automatizado (de propósito); reprodução em `scripts/reproduzir_bug_023.py`

Sem teste automatizado, de propósito: é um bug de tempo; um `xfail(strict=True)` passaria por acaso em parte das execuções e deixaria o CI instável.

Como foi descoberto: ao paralelizar a suíte com pytest-xdist (vários navegadores ao mesmo tempo, máquina sob carga), o teste do Reset App State passou a falhar de forma intermitente, em 3 de 10 execuções completas. O clique em "Open Menu" era aceito, mas o link do menu nunca ficava visível. O trace do Playwright mostrou o menu num estado contraditório: `aria-hidden="false"` (aberto para o componente) e `hidden="true"` (invisível). A leitura do código do componente de menu do site (`react-burger-menu`) isolou a causa no site, não no teste.

Causa: ao carregar a página (e ao fechar o menu), o componente agenda um timer de 500 ms que esconde o menu fechado (atributo `hidden`). O clique em "Open Menu" remove o `hidden` na hora, mas só marca o menu como aberto numa tarefa seguinte, que é quando o timer é cancelado. Se a máquina está lenta e o clique cai pouco antes dos 500 ms, o timer dispara no meio desse caminho e esconde o menu que acabou de abrir.

Pré-condições: usuário logado; máquina lenta (reproduzido deixando a CPU do navegador 30x mais lenta pelas DevTools).

Passos

1. Abrir o inventário.
2. Clicar em **Open Menu** entre ~300 e ~450 ms depois de a página aparecer.

Esperado: o menu lateral abre.

Obtido: o menu não aparece; o componente fica com `aria-hidden="false"` e `hidden="true"`.

Reprodutibilidade (script de reprodução controlada, 2 rodadas):

Condição	Menu invisível
CPU 30x mais lenta, clique entre 300 e 450 ms	32 de 36 tentativas (89%)
CPU 30x mais lenta, clique entre 480 e 540 ms	0 de 18 (0%)
CPU 30x mais lenta, total	32 de 54 (59%)
CPU normal (controle), mesmos atrasos	1 de 54 (2%), com o clique a 450 ms

Efeito na suíte e contorno: como os testes de menu clicam logo depois de a página carregar, a falha aparecia sob carga. O `Header.open_menu()` agora espera o atributo `hidden` aparecer no menu fechado antes de clicar. Esse é o sinal de que o timer de 500 ms já rodou e o clique é seguro. Com o contorno, a suíte paralela passou em 10 de 10 execuções. O bug do site continua lá; a suíte só deixou de tropeçar nele.

Evidência: resultado da reprodução controlada · script de reprodução

BUG-024: Resumo do checkout abre pela URL e conclui o pedido sem o passo 1

Usuário	Severidade	Prioridade	Teste
todos (`standard_user`)	Média	P2	`tests/test_cart_checkout.py::test_resumo_exige_o_passo_1_preenchido`

Como foi descoberto: o teste do BUG-013 já ia direto ao resumo para contornar o formulário quebrado do `problem_user` (BUG-009). O contorno só funciona porque o site não exige o passo 1, o que nunca tinha sido registrado como defeito. Encontrado na comparação com o projeto em Cypress, que tem um bug parecido (checkout e pagamento abertos pela URL).

Pré-condições: usuário logado, com pelo menos um item no carrinho.

Passos

1. Adicionar um produto ao carrinho e abrir o carrinho (`/cart.html`).
2. Em vez de clicar em **Checkout**, digitar na barra de endereço `/checkout-step-two.html`.
3. Clicar em **Finish**.

Esperado: sem nome, sobrenome e CEP (RN-15), o resumo não deve abrir: o site deve voltar ao passo 1 ou bloquear o **Finish**.

Obtido: o resumo abre com os itens, o pagamento, a entrega e o total, e o pedido é concluído com "Thank you for your order!". Os dados do comprador não existem em lugar nenhum (nem no `localStorage` nem no `sessionStorage`). O recibo gerado por **Generate PDF order** confirma o pedido

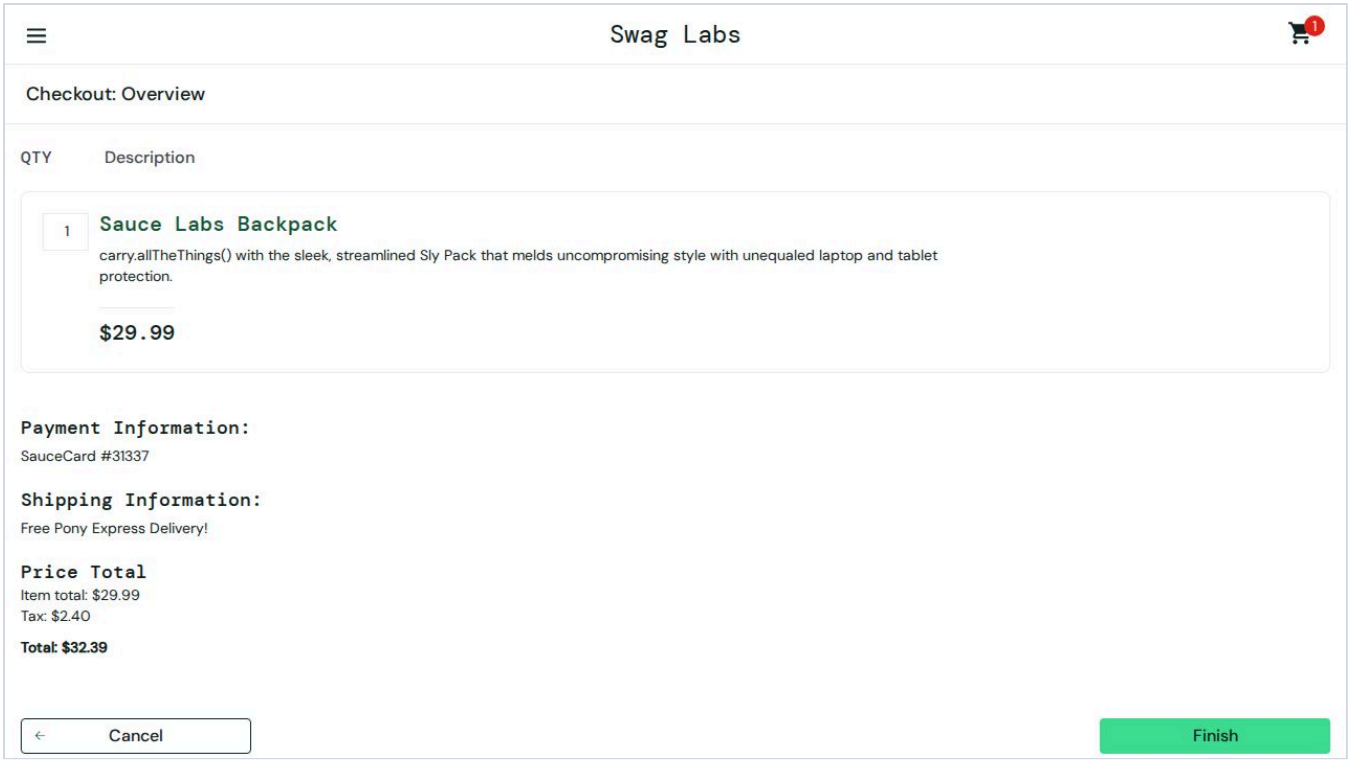
como "dispatched" sem a seção **SHIP TO**: não há nome nem endereço de entrega (no fluxo normal, o recibo traz nome, sobrenome e CEP).

Reprodutibilidade: 100%, em 2 execuções independentes, com `standard_user`, `performance_glitch_user` e `visual_user`.

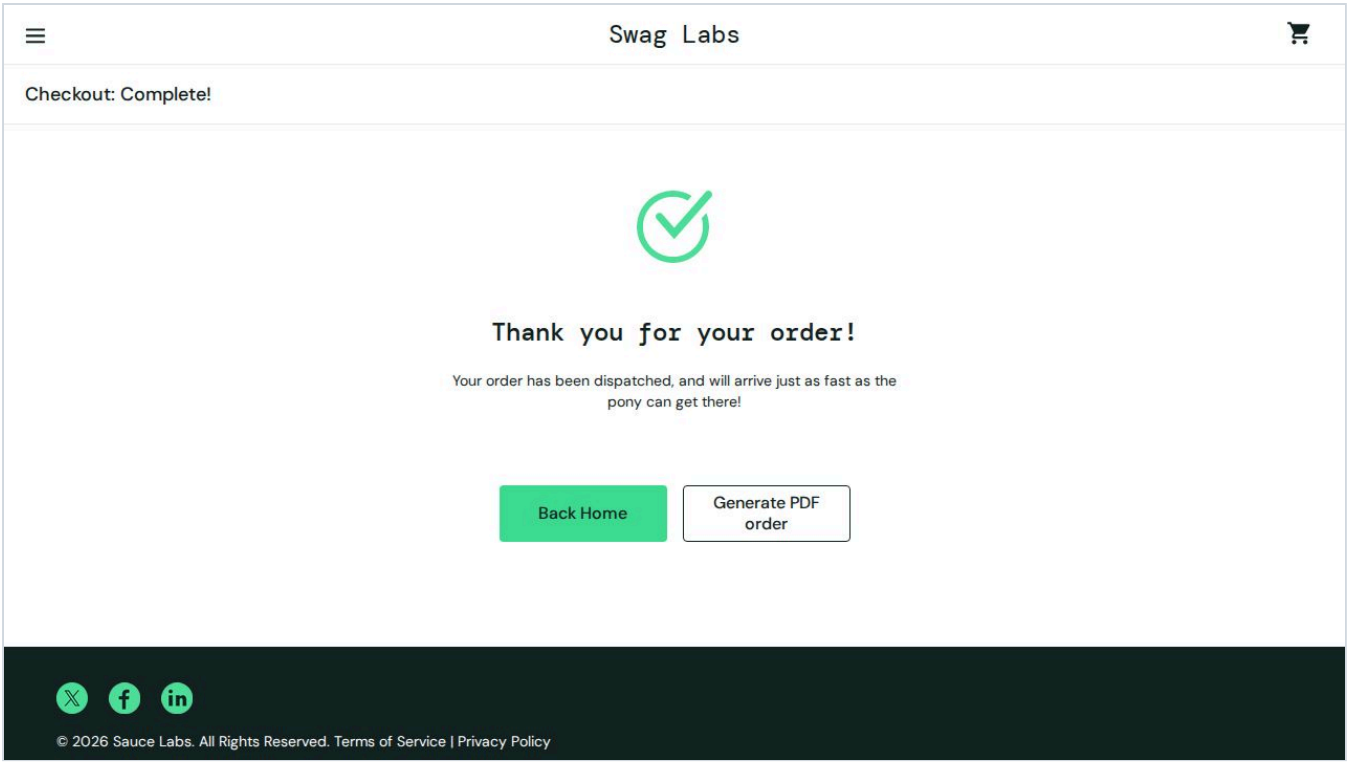
Efeito na suíte: quatro testes abrem o resumo direto pela URL para isolar o que verificam e, por isso, dependem deste bug. Se ele for corrigido, eles passam a cair no passo 1 e precisam preenchê-lo antes:

- `test_cart_checkout.py::test_resumo_calcula_subtotal_imposto_e_total`;
- `test_cart_checkout.py::test_concluir_o_pedido_esvazia_o_carrinho`;
- `test_problem_user.py::test_carrinho_esvazia_apos_concluir_o_pedido` (BUG-013);
- `test_error_user.py::test_finish_conclui_o_pedido` (BUG-018).

Evidência: resumo aberto sem o passo 1 · pedido concluído sem os dados · recibo sem dados de entrega · recibo normal, para comparação



Evidência: resumo aberto sem o passo 1



Evidência: pedido concluído sem os dados

Bugs do problem_user

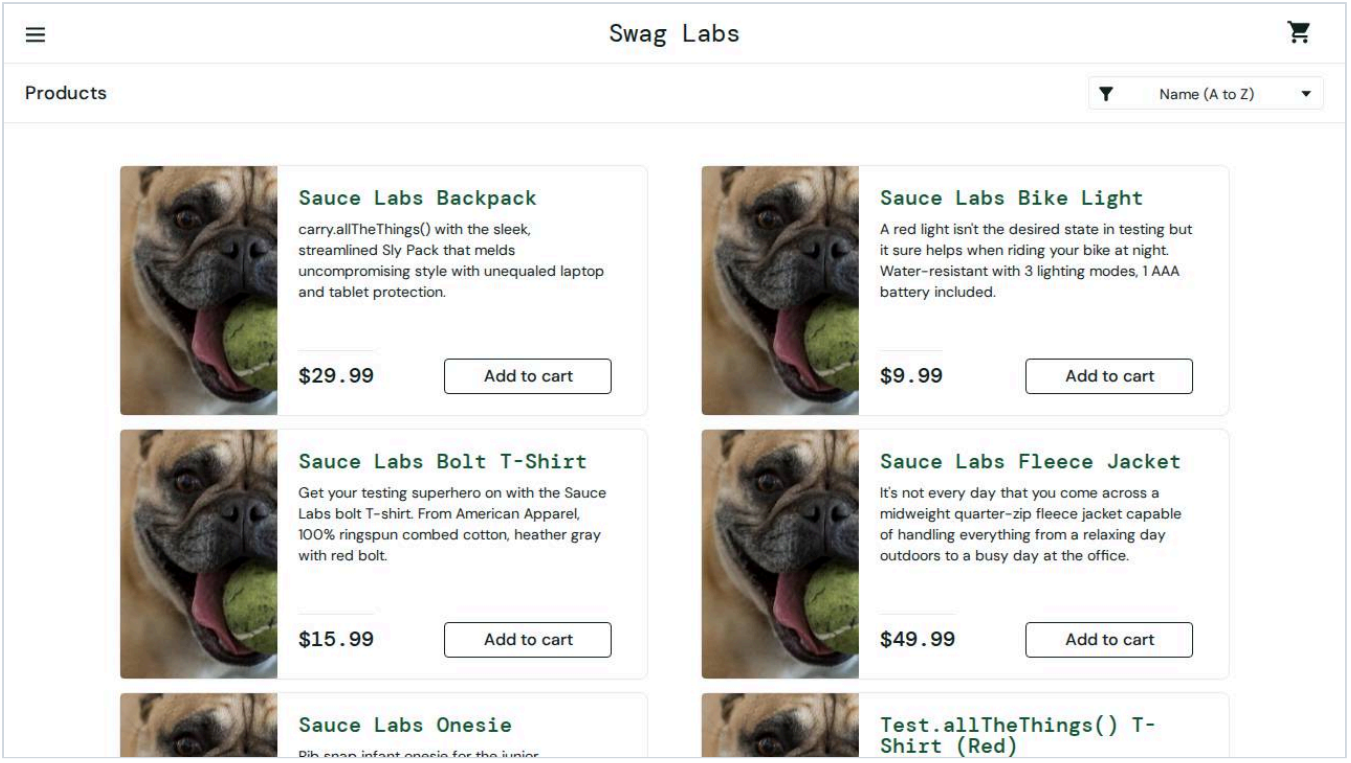
BUG-007: Todos os produtos exibem a imagem de erro

Severidade	Prioridade	Teste
Média	P3	`tests/test_inventory.py::test_cada_produto_exibe_sua_propria_imagem[problem_user]` (já existia)

Passos: fazer login como `problem_user`.

Esperado: cada produto com a sua imagem. **Obtido:** os 6 produtos exibem a mesma imagem `sl-404` (um cachorro).

Evidência: inventário do problem_user



Evidência: inventário do problem_user

BUG-008: Ordenação não muda a lista

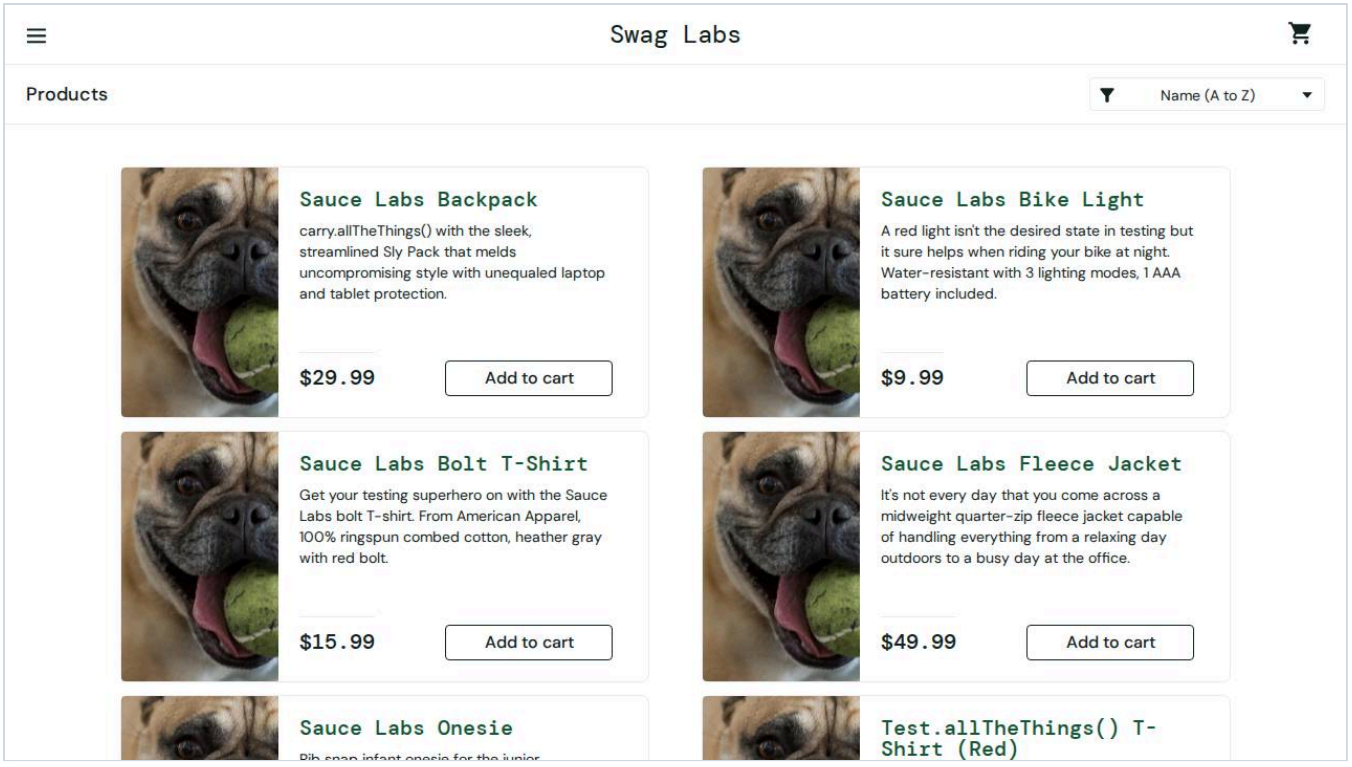
Severidade	Prioridade	Teste
Média	P2	`tests/test_problem_user.py::test_ordenacao_por_nome_z_a`

Passos: no inventário, escolher "Name (Z to A)" (ou qualquer outra ordenação).

Esperado: a lista reordenada e a opção escolhida selecionada.

Obtido: a lista não muda e o seletor volta para "Name (A to Z)".

Evidência: lista ainda em A–Z



Evidência: lista ainda em A–Z

BUG-009: Sobrenome digitado vai para o campo de nome

Severidade	Prioridade	Teste
Crítica	P1	`tests/test_problem_user.py::test_sobrenome_fica_no_campo_de_sobrenome`

Impacto (Crítica): o `problem_user` não consegue concluir nenhuma compra, porque o formulário nunca aceita o sobrenome.

Pré-condições: item no carrinho, no passo 1 do checkout.

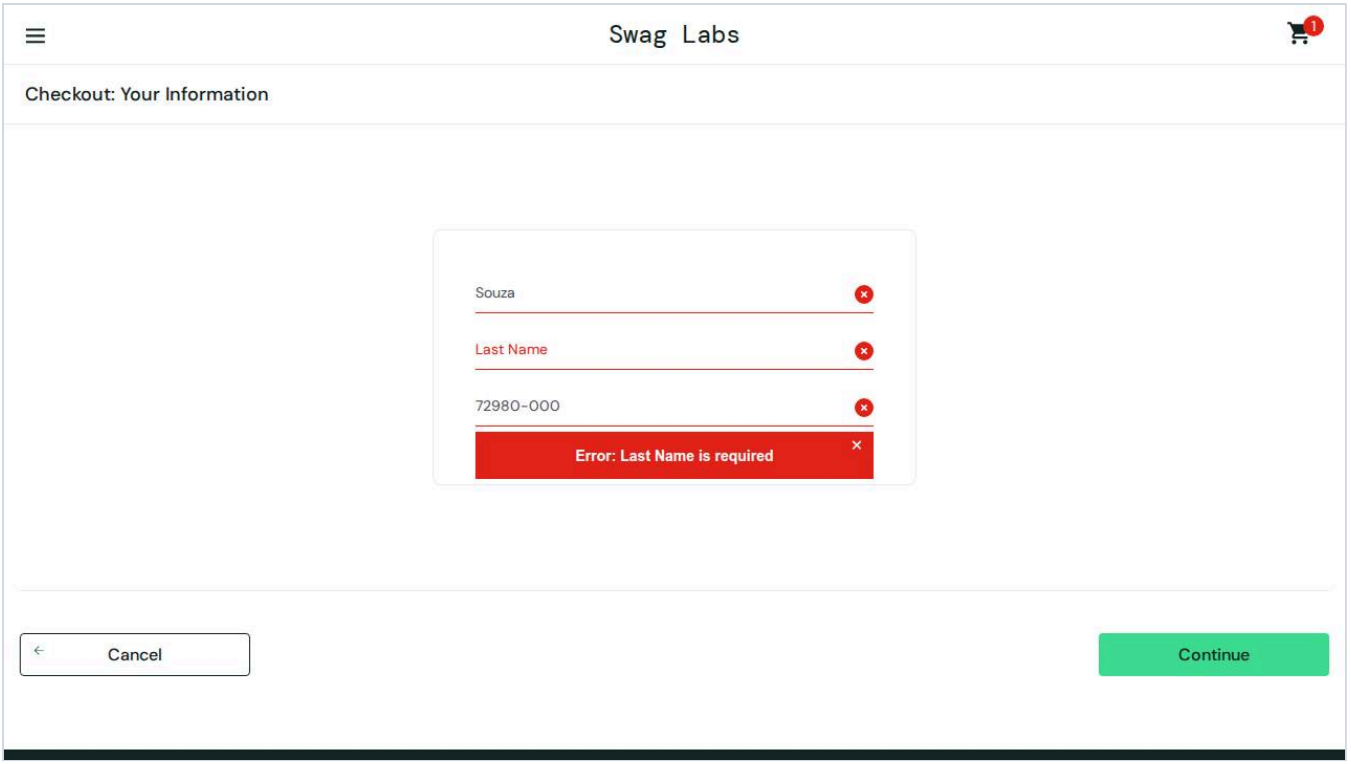
Passos

1. Preencher nome "Ana", sobrenome "Souza" e CEP.
2. Clicar em **Continue**.

Esperado: avançar para o resumo.

Obtido: o texto do sobrenome substitui o nome ("Souza" no campo de nome) e o sobrenome fica vazio. Digitando tecla a tecla, cada letra substitui a anterior no campo de nome (fica só "a"). O erro "Last Name is required" **impede concluir a compra**.

Evidência: nome com "Souza" e sobrenome vazio



Evidência: nome com "Souza" e sobrenome vazio

BUG-010: Adicionar falha em 3 produtos e remover falha nos outros 3

Severidade	Prioridade	Teste
Alta	P1	`tests/test_problem_user.py::test_adicionar_e_remover_cada_produto` (6 casos)

Impacto (Alta): metade do catálogo não pode ser comprada, e o que entra no carrinho não sai, o que leva a compras indesejadas.

Passos: para cada produto, clicar em **Add to cart** e depois em **Remove**, no inventário ou no detalhe.

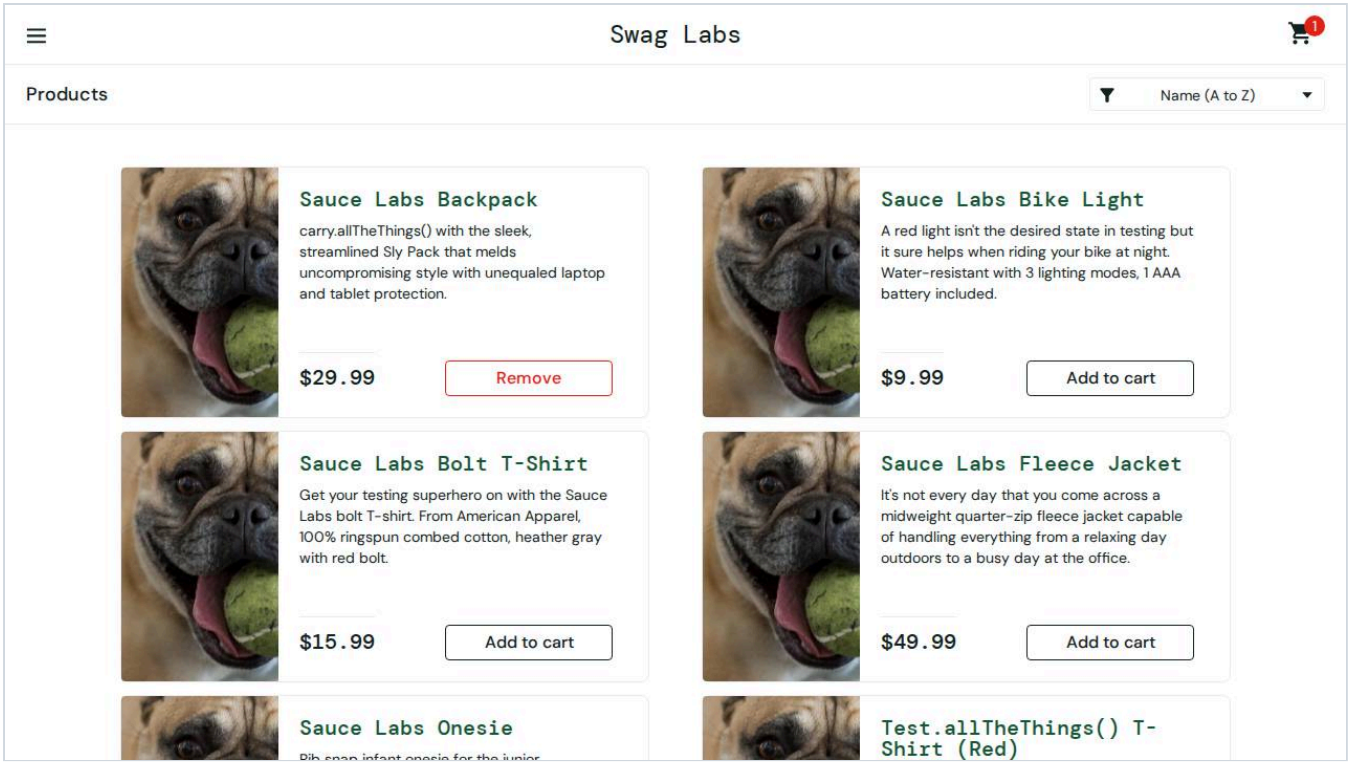
Esperado: o produto entra e sai do carrinho; contador e botão acompanham.

Obtido:

- Bolt T-Shirt, Fleece Jacket e T-Shirt (Red): o clique em **Add to cart** não faz nada.
- Backpack, Bike Light e Onesie: entram no carrinho, mas o clique em **Remove** não faz nada.

A falha é silenciosa, sem nenhuma mensagem.

Evidência: Bolt sem entrar; Backpack sem sair



Evidência: Bolt sem entrar; Backpack sem sair

BUG-011: Detalhe abre o produto seguinte ao clicado

Severidade	Prioridade	Teste
Média	P2	`tests/test_problem_user.py::test_detalhe_mostra_o_produto_clicado`

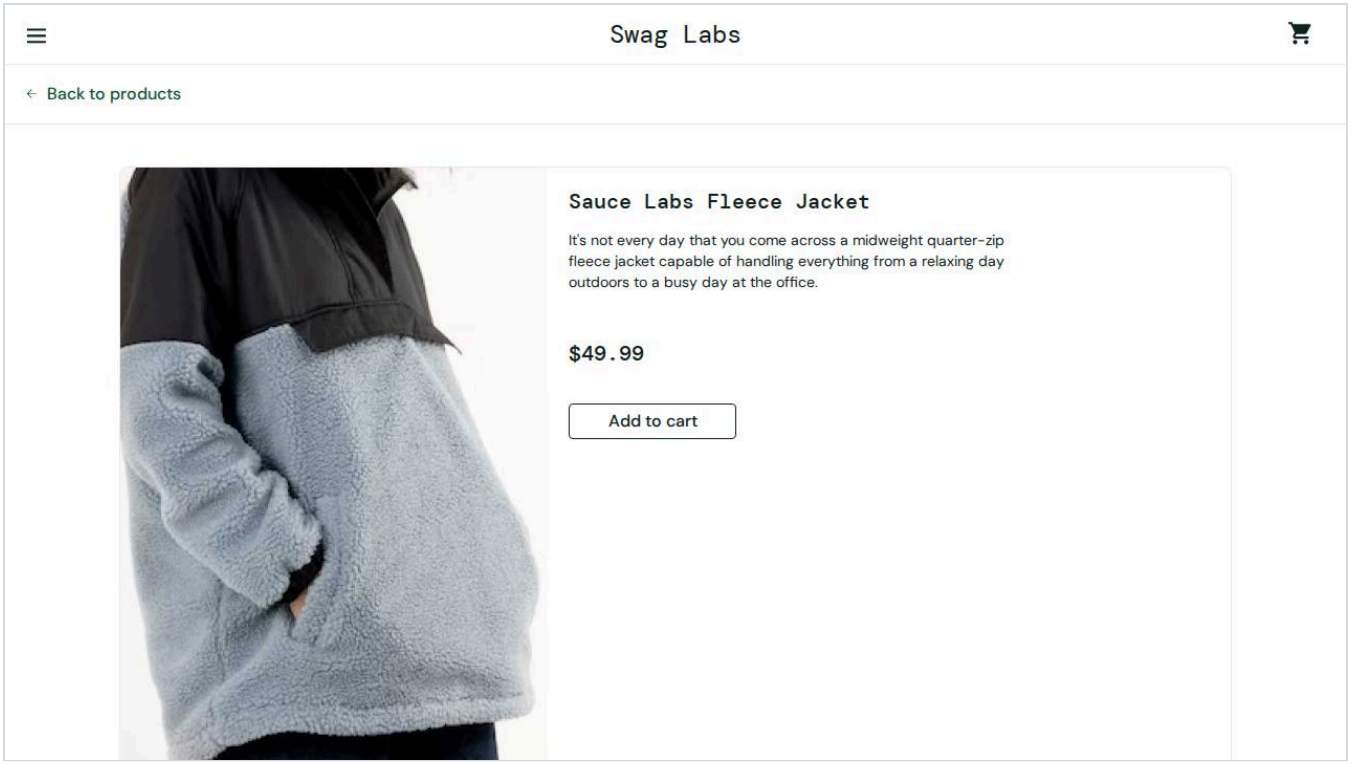
Severidade revisada (Alta → Média): a tela exibe nome e preço coerentes com o produto aberto e nenhum valor errado é cobrado; é um erro de navegação, não de compra.

Passos: no inventário, clicar no nome ou na imagem de um produto.

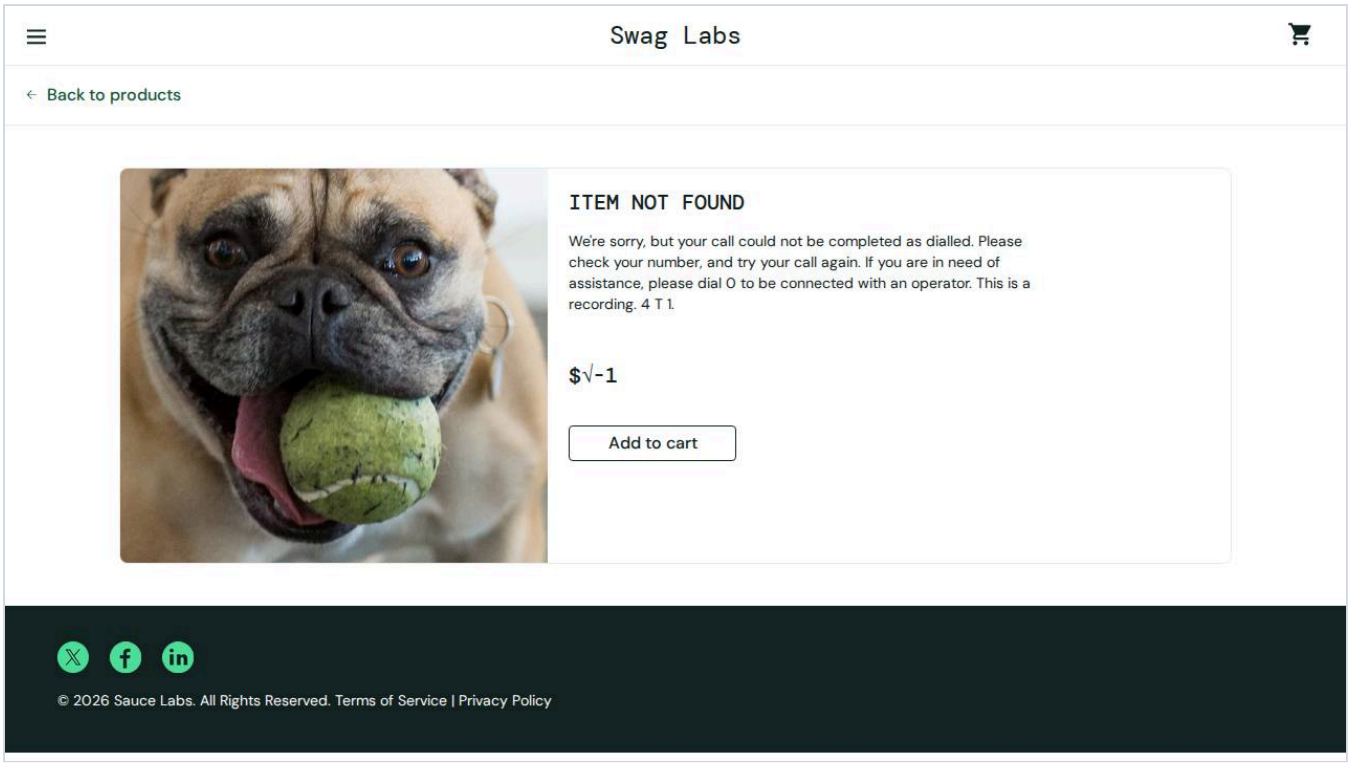
Esperado: o detalhe do produto clicado (ex.: Backpack, id 4).

Obtido: o link usa o id seguinte: Backpack abre o Fleece Jacket (id 5), Bike Light abre o Bolt T-Shirt (id 1), e assim por diante. O Fleece Jacket abre o id 6, que não existe ("ITEM NOT FOUND").

Evidência: clicou na Backpack e abriu o Fleece Jacket · clicou no Fleece Jacket e abriu ITEM NOT FOUND



Evidência: clicou na Backpack e abriu o Fleece Jacket



Evidência: clicou no Fleece Jacket e abriu ITEM NOT FOUND

BUG-012: Link About aponta para uma página 404

Severidade	Prioridade	Teste
Baixa	P3	`tests/test_menu.py::test_link_about_aponta_para_a_sauce_labs[problem_user]`

Passos: abrir o menu e verificar o destino de **About**.

Esperado: `https://saucelabs.com/`, como no `standard_user`. **Obtido:** `https://saucelabs.com/error/404`, que responde HTTP 404.

Evidência: logs-rede-console-e-estado.json (chave "BUG-012")

BUG-013: Carrinho continua cheio depois do pedido

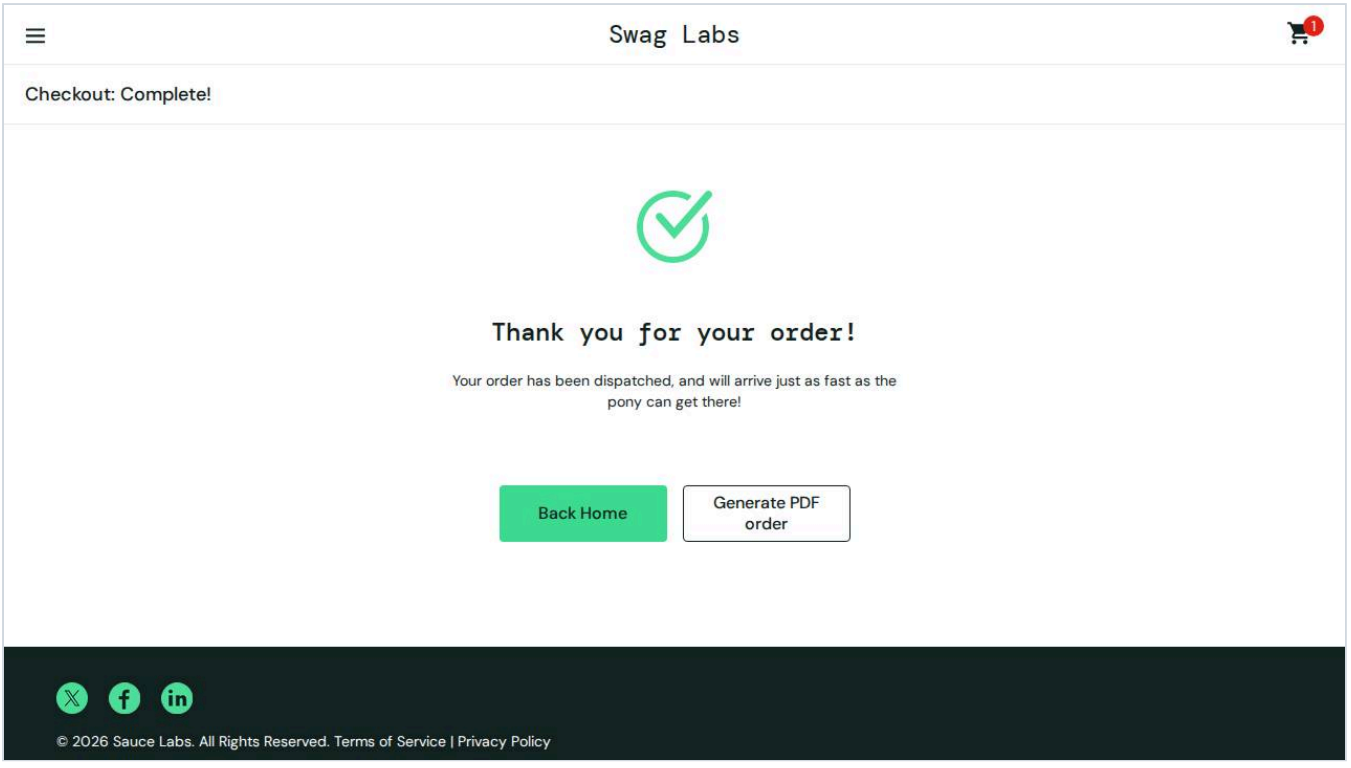
Severidade	Prioridade	Teste
Média	P2	`tests/test_problem_user.py::test_carrinho_esvazia_apos_concluir_o_pedido`

Pré-condições: Backpack no carrinho. Como o formulário está quebrado (BUG-009), o passo 1 foi pulado indo direto a `/checkout-step-two.html`. Esse contorno só funciona por causa do BUG-024: se ele for corrigido, este teste terá de preencher o passo 1 de outra forma.

Passos: clicar em **Finish**.

Esperado: pedido concluído e carrinho vazio (RN-19). **Obtido:** pedido concluído, mas o contador continua em 1 e o item fica no `localStorage`.

Evidência: pedido concluído com contador 1



Evidência: pedido concluído com contador 1

Bugs do error_user

BUG-014: Ordenação exibe "Sorting is broken!" e não ordena

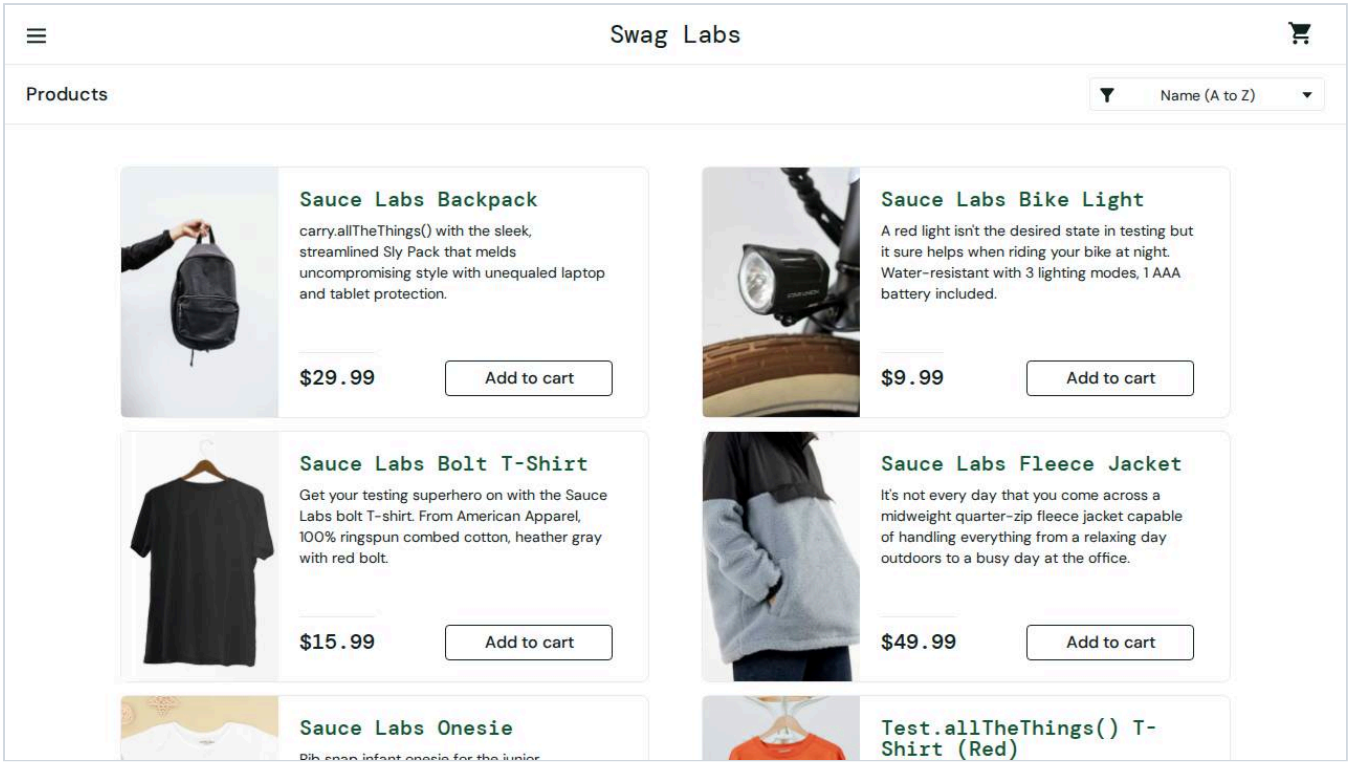
Severidade	Prioridade	Teste
Média	P2	`tests/test_error_user.py::test_ordenacao_por_nome_z_a`

Passos: escolher qualquer ordenação no inventário.

Esperado: a lista reordenada.

Obtido: aparece um alerta nativo "Sorting is broken! This error has been reported to Backtrace.", a lista não muda e o seletor volta para A–Z. O erro também não chega ao Backtrace (BUG-006).

Evidência: lista inalterada · texto do alerta (chave "BUG-014")



Evidência: lista inalterada

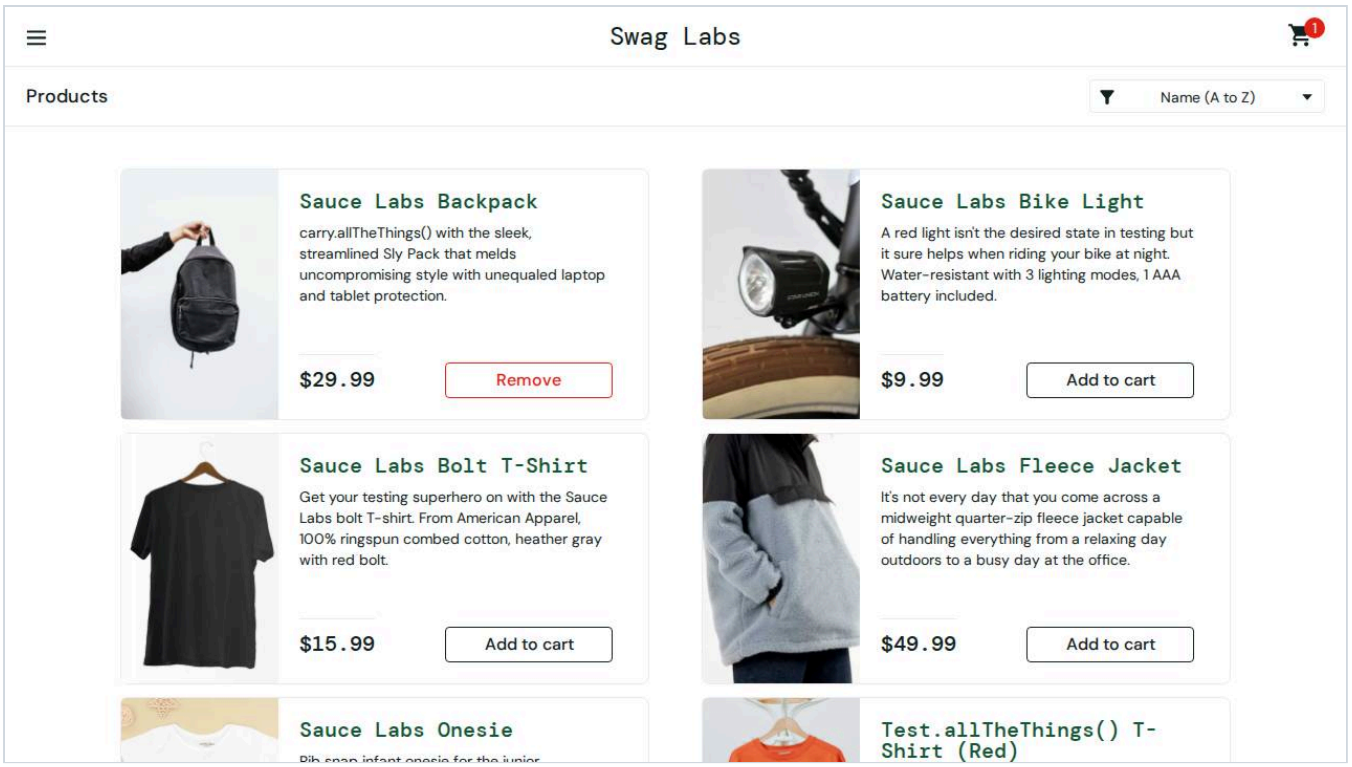
BUG-015: Adicionar falha em 3 produtos e remover falha nos outros 3, com erro JavaScript

Severidade	Prioridade	Teste
Alta	P1	`tests/test_error_user.py::test_adicionar_e_remover_cada_produto` (6 casos)

Impacto (Alta): o mesmo do BUG-010: metade do catálogo não pode ser comprada e o que entra no carrinho não sai.

Passos e obtido: os mesmos produtos do BUG-010 falham, mas aqui cada falha lança um erro JavaScript não tratado ("Failed to add item to the cart." / "Failed to remove item from cart."). O usuário não recebe nenhuma mensagem.

Evidência: botões · erros JavaScript (chave "BUG-015")



Evidência: botões

BUG-016: Campo de sobrenome não aceita texto

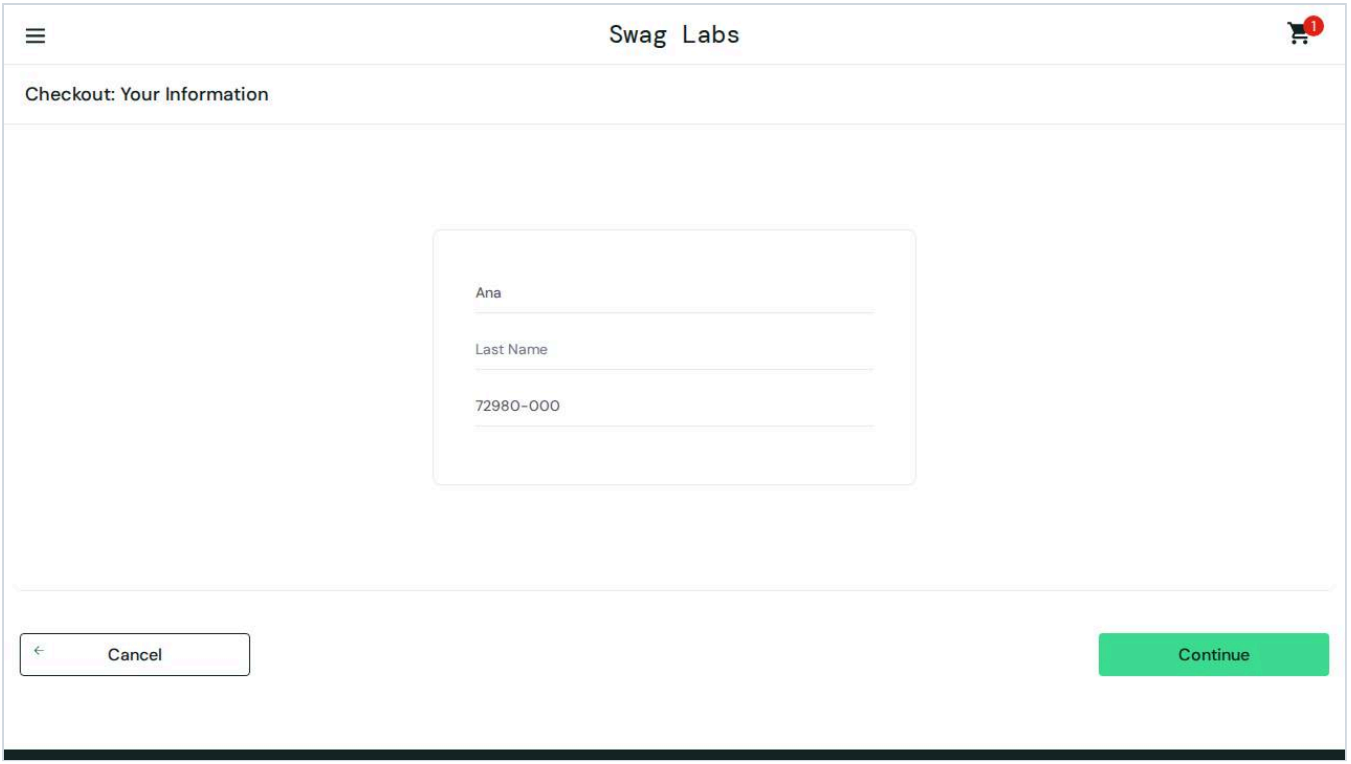
Severidade	Prioridade	Teste
Média	P2	`tests/test_error_user.py::test_campo_sobrenome_aceita_texto`

Severidade revisada (Alta → Média): o pedido segue sem sobrenome (dado incompleto), mas a compra não é bloqueada por isso; o bloqueio vem do BUG-018.

Passos: no passo 1 do checkout, digitar "Souza" no sobrenome.

Esperado: o campo exibe "Souza". **Obtido:** o campo continua vazio, tanto com preenchimento direto quanto tecla a tecla.

Evidência: sobrenome vazio após digitar



Evidência: sobrenome vazio após digitar

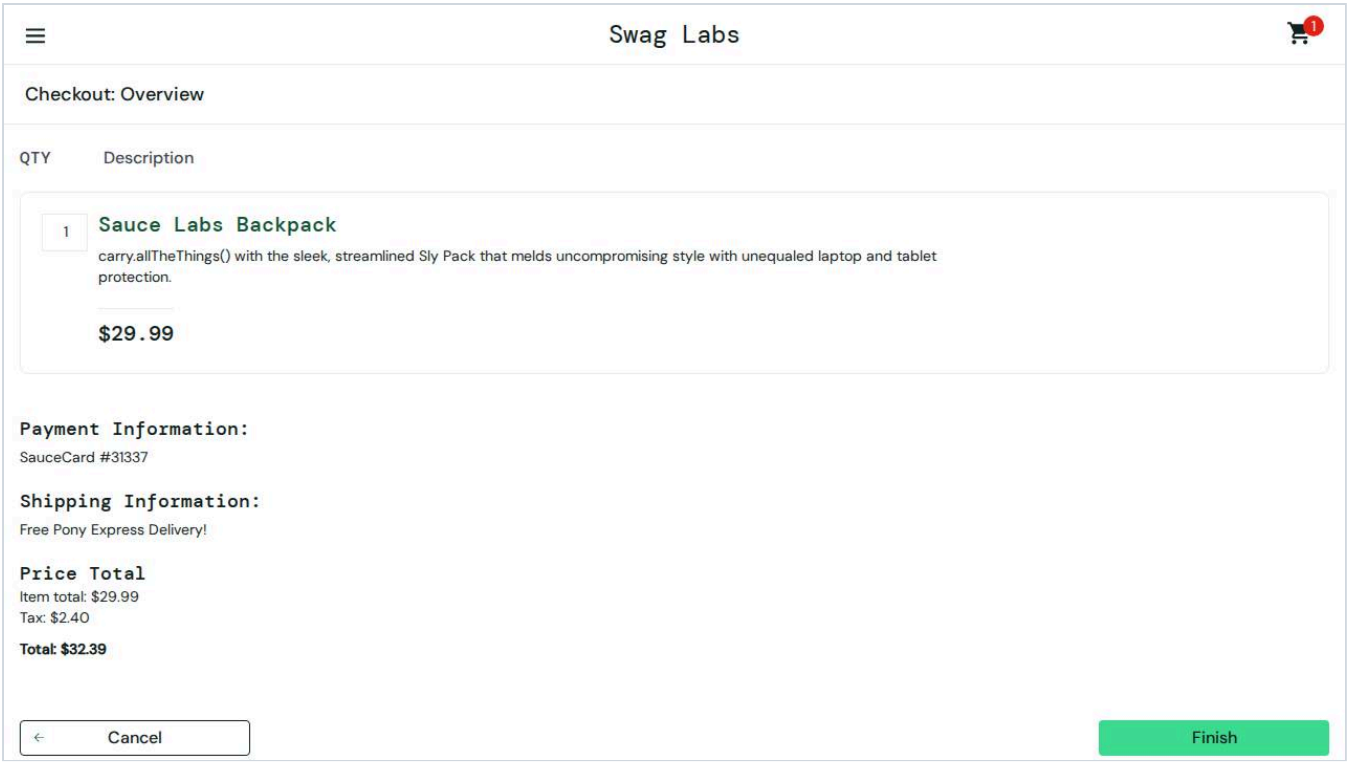
BUG-017: Checkout avança sem sobrenome

Severidade	Prioridade	Teste
Média	P2	`tests/test_error_user.py::test_checkout_exige_sobrenome`

Passos: preencher só nome e CEP e clicar em **Continue**.

Esperado: "Error: Last Name is required" (RN-15), como no `standard_user`. **Obtido:** avança para o resumo sem sobrenome. Combinado ao BUG-016, o pedido segue sem esse dado.

Evidência: resumo sem sobrenome



Evidência: resumo sem sobrenome

BUG-018: Finish não conclui o pedido

Severidade	Prioridade	Teste
Crítica	P1	`tests/test_error_user.py::test_finish_conclui_o_pedido`

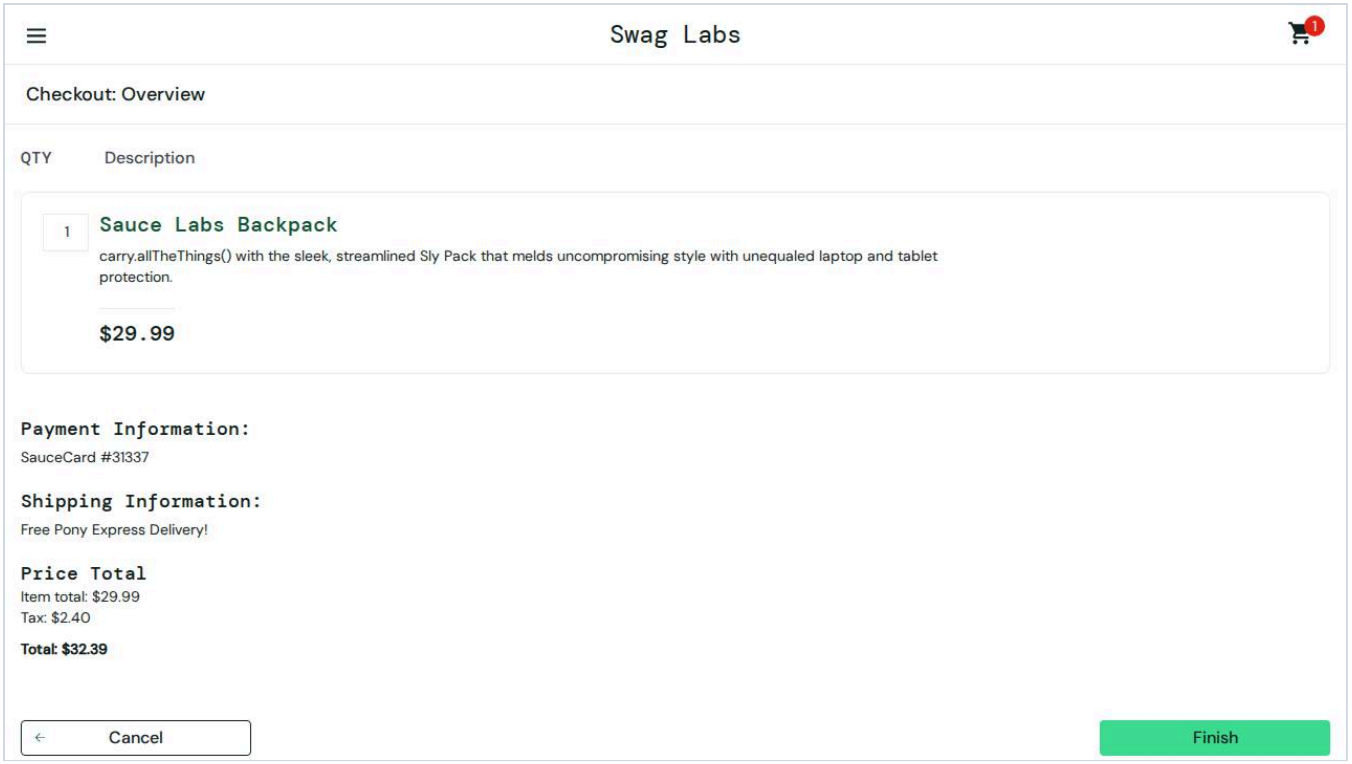
Impacto (Crítica): o `error_user` não consegue concluir nenhuma compra; o último passo do fluxo não faz nada.

Pré-condições: item no carrinho, no resumo (passo 2).

Passos: clicar em **Finish**.

Esperado: página "Thank you for your order!". **Obtido:** nada acontece na tela; o usuário continua no resumo, sem mensagem. O clique lança o erro JavaScript "La.cesetRart is not a function". **A compra não pode ser concluída.**

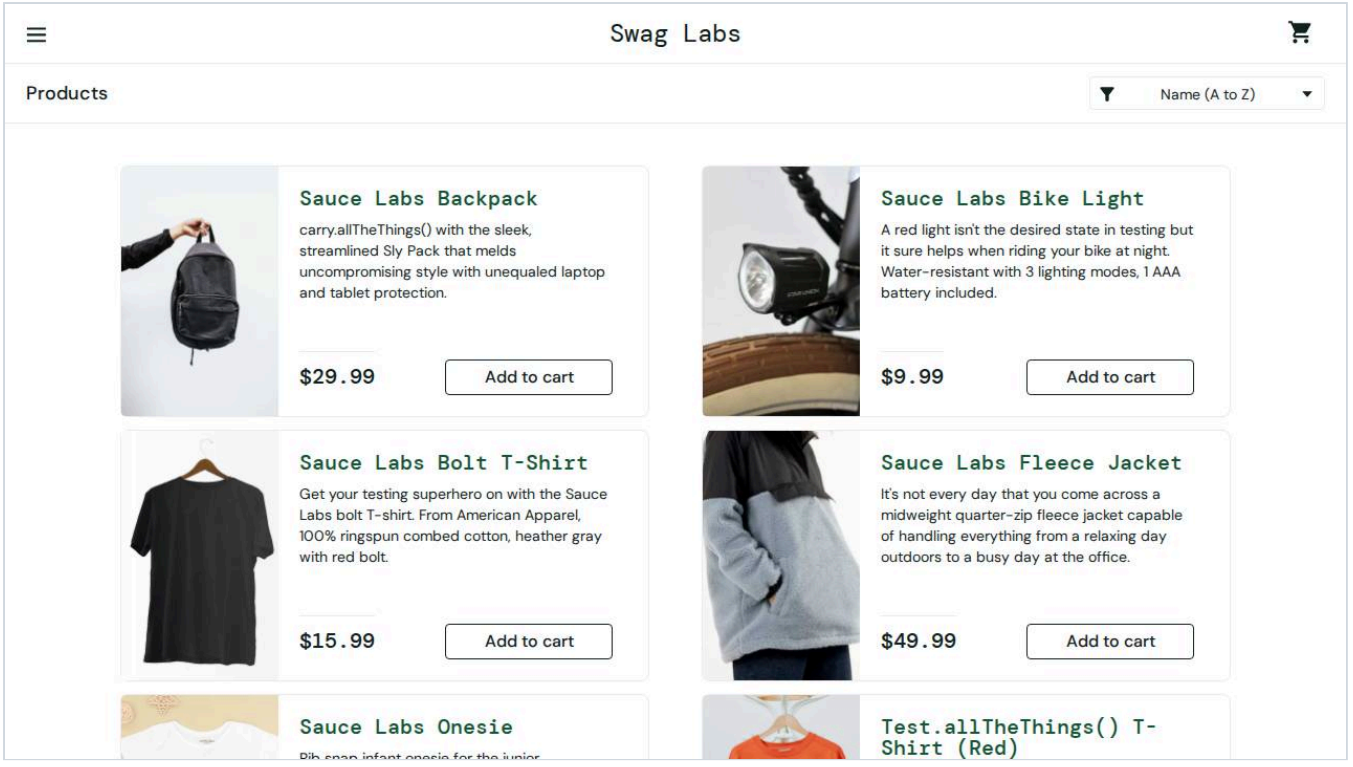
Evidência: continua no resumo · erros JavaScript (chave "BUG-018")



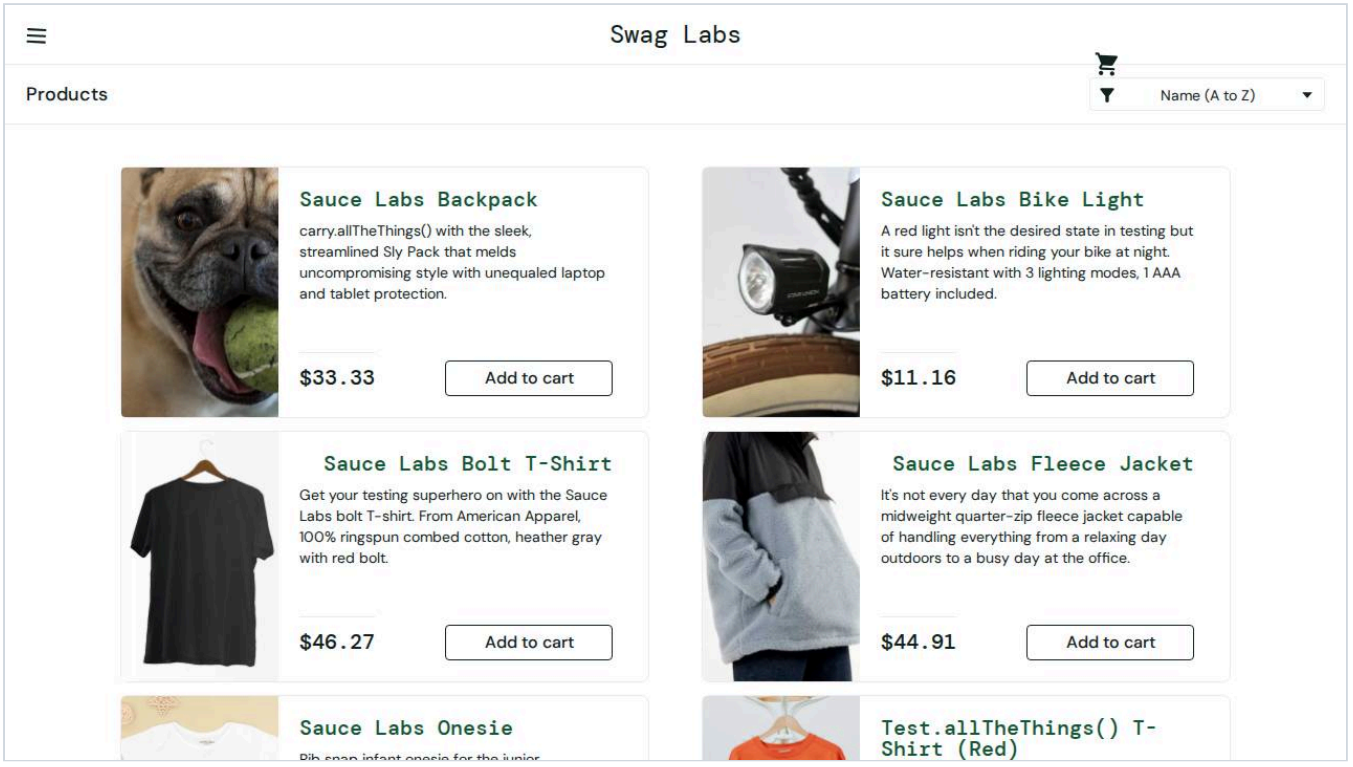
Evidência: continua no resumo

Bugs do visual_user

Os testes deste perfil usam o `standard_user` como **oráculo**: os valores esperados são lidos dele durante o teste. Posição e tamanho dos elementos usam tolerância de 5 px, após o carregamento das fontes web. Comparação: inventário standard_user · inventário visual_user.



Evidência: inventário standard_user



Evidência: inventário visual_user

BUG-019: Preços do inventário aleatórios e diferentes do cobrado

Severidade	Prioridade	Teste
Alta	P1	`tests/test_visual_user.py::test_precos_iguais_aos_do_standard_user`

Impacto (Alta): o cliente decide a compra vendo um preço e paga outro, o que gera reclamações e estornos.

Passos

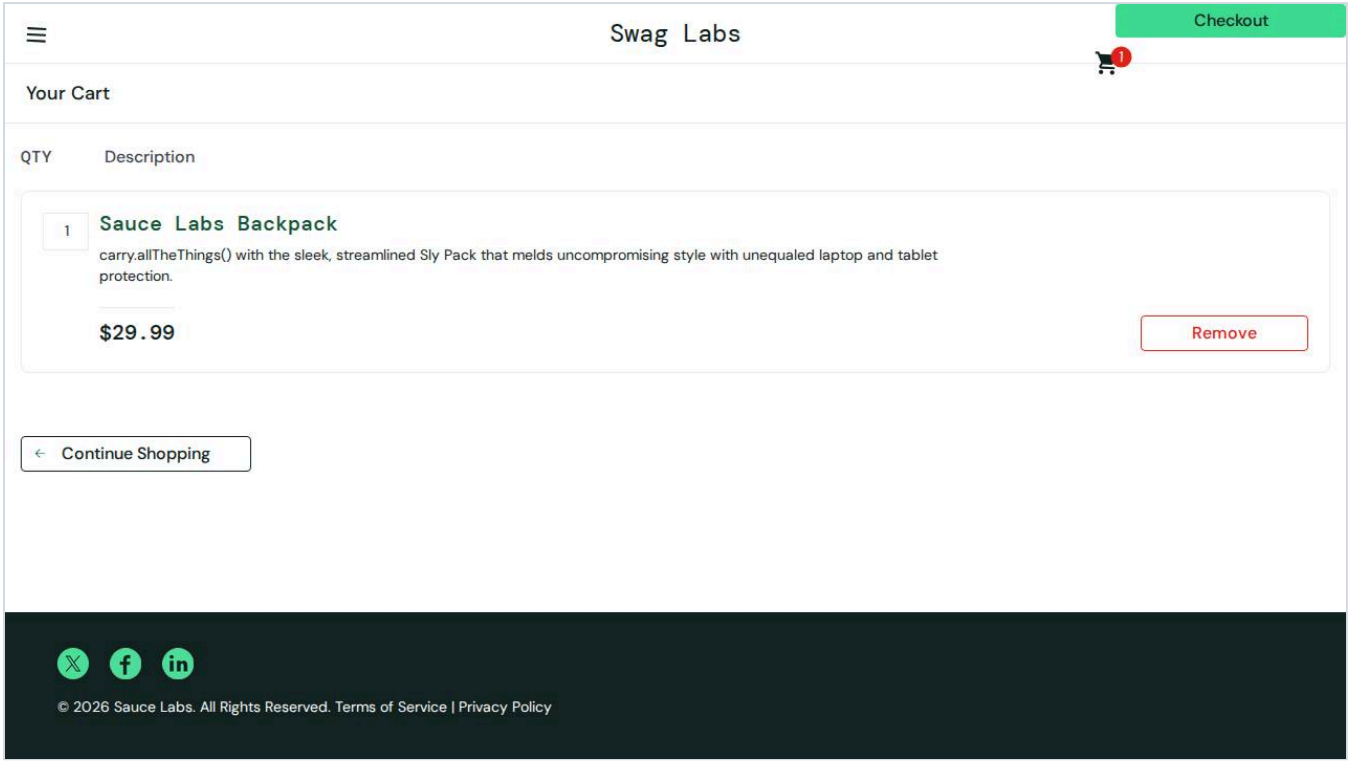
1. Fazer login como `visual_user` e anotar o preço da Backpack.
2. Recarregar a página e comparar o preço.
3. Clicar em **Add to cart** na Backpack.
4. Abrir o carrinho.
5. Comparar o preço do carrinho com o anotado no passo 1.

Esperado: $29.99 em todas as telas, como para o `standard_user`.

Obtido:

- O inventário mostra um preço diferente a cada carregamento para **todos os 6 produtos**, e nenhum coincide com o catálogo (em 3 carregamentos seguidos, a Backpack apareceu a $21.45, $98.47 e $67.68). Alguns valores perdem o zero final ($0.5, $66.2).
- O carrinho, o detalhe e o resumo mostram o preço real, $29.99, que é o valor cobrado.
- A ordenação por preço parece errada, porque usa os valores reais e não os exibidos.

Evidência: preços em 3 carregamentos seguidos · carrinho com $29.99 · preço do inventário x carrinho (chave "BUG-019")



Evidência: carrinho com $29.99

BUG-020: Backpack exibe a imagem de erro

Severidade	Prioridade	Teste
Baixa	P3	`tests/test_visual_user.py::test_imagens_iguais_as_do_standard_user`

Obtido: no inventário, a Backpack exibe `sl-404` (cachorro) em vez da mochila; os outros 5 produtos estão corretos. No detalhe, a imagem está certa.

BUG-021: Ícone do carrinho deslocado sobre a ordenação

Severidade	Prioridade	Teste
Média	P3	`tests/test_visual_user.py::test_layout_igual_ao_do_standard_user[icone-do-carrinho]`

Obtido: o ícone do carrinho fica 186 px à esquerda e 29 px abaixo da posição do `standard_user`, colado ao seletor de ordenação e fora do canto superior direito, onde o usuário o procura.

BUG-022: Botão do último produto desalinhado

Severidade	Prioridade	Teste
Baixa	P4	`tests/test_visual_user.py::test_layout_igual_ao_do_standard_user[botao-do-ultimo-produto]`

Obtido: o botão "Add to cart" da T-Shirt (Red) fica 54 px à direita dos demais e ultrapassa a borda do card.

Suspeitas

Comportamentos reproduzidos, mas cujo impacto não pôde ser confirmado no site de demonstração (não há back-end para verificar pedidos, por exemplo). Não viraram teste `xfail`.

ID	Observação	Por que é suspeita
S-01	Depois de concluir o pedido, o botão **Voltar** do navegador reexibe o resumo com o item, e **Finish** conclui o pedido de novo.	Em uma loja real, permitiria pedido duplicado; aqui não há registro de pedidos para confirmar.
S-02	O menu tem o item **Dynamic Catalog** (`href="#"`), que não produz efeito visível para nenhum usuário.	Pode ser funcionalidade incompleta ou dependente de uma configuração não identificada.
S-03	Com o `localStorage` adulterado, um id repetido (`[4, 4]`) aparece duas vezes e é cobrado duas vezes ($59.98 + imposto). Um valor que não é JSON gera erro JavaScript não tratado.	Exige adulterar o estado do navegador; a interface não permite repetir itens.
S-04	No celular (390×844), o botão do menu mede 20×20 px, abaixo do mínimo de 24×24 px do WCAG 2.2 (critério 2.5.8).	O axe-core não acusou, e o critério aceita alvos menores quando há espaço livre suficiente ao redor; precisa de avaliação manual.

Comportamento intencional do site de demonstração

Observado, mas **não** é defeito:

Comportamento	Detalhe
Lentidão do `performance_glitch_user`	Atraso fixo de ~5 s toda vez que o inventário é renderizado (login, ordenação e volta ao inventário). Medições em requisitos-inferidos.md.
Bloqueio do `locked_out_user`	Recusado no login com mensagem clara. Também é recusado se o cookie for forjado com esse usuário.
Página "ITEM NOT FOUND"	O texto humorístico e o preço `$√-1` para um id inexistente são parte do site; o defeito está apenas em permitir adicioná-lo ao carrinho (BUG-004).
Sessão no cookie em texto puro	`session-username` guarda o nome do usuário e vale 10 minutos. É a forma como o site de demonstração simula login, sem back-end.
Pagamento e entrega fixos	"SauceCard #31337" e "Free Pony Express Delivery!" para qualquer pedido.

Verificações sem achados: nenhuma violação **séria ou crítica** do axe-core 4.10 nas 6 telas (login, inventário, detalhe, carrinho e os dois passos do checkout); nenhuma rolagem horizontal em 390×844; nenhuma exceção JavaScript no fluxo completo de compra do `standard_user`.

Confirmação aberta pela URL (verificado em 27/10/2026, sem achado): logado e com o carrinho `[4, 5]` (montado pelo `localStorage` e, em outra execução, pelos cliques na tela), abrir `/checkout-complete.html` direto mostra "Thank you for your order!", mas **nada muda**: o carrinho continua `[4, 5]`, inclusive depois de recarregar, e o botão **Generate PDF order** não aparece (ele só existe depois de um **Finish** de verdade). Ao contrário do projeto em Cypress, em que a confirmação e a fatura eram geradas pela URL, aqui a página é só uma tela estática, sem pedido. Sem login, a rota redireciona para o login (coberta em `test_rotas_protegidas_exigem_login`).